



Institute of Distance and Open Learning

Vidya Nagari, Kalina, Santacruz East – 400098.

CERTIFICATE

This is to certify that Mr/Ms. _____ of **Master in Computer Application** (MCA) Semester II has completed the specified term work in the subject _____ of satisfactorily within this institute as laid down by University of Mumbai during the academic year

Subject In-charge

External Examiner

Coordinator – M.C.A

Index

Sr No.	Practical Topics	Sign
1	Basic Git commands	
2	Create a Fork Repository in GitHub and Apply Branch, Merge, and Rebase Concepts	
3	Using Git for Collaboration	
4	Collaboration and Cloning Using GitHub	
5	Using GitLab Web IDE	
6	Performing Merge Request Using GitLab	
7	Workflow Management in GitLab	
8	Demonstrate Continuous Integration and Continuous Deployment (CI/CD) Using Jenkins	
9	Explore Docker Commands for Container Management	
10	Develop a Simple Containerized Application Using Docker	
11	Ad-Hoc Ansible Commands	
12	Using Ansible Playbooks	

Practical 1: Basic Git commands

Commands:

Configure Git

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

Output:

(no output) (configuration saved)

Check Git version

```
git --version
```

Output:

git version 2.45.1

Initialize a Git repository

```
git init
```

Output:

Initialized empty Git repository in C:/Project/.git/

Check repository status

```
git status
```

Output:

On branch master

No commits yet

nothing to commit

Add a specific file

```
git add filename
```

Add all files

```
git add .
```

Commit changes

```
git commit -m "Initial commit"
```

Output:

[master (root-commit) a1b2c3d] Initial commit

1 file changed, 1 insertion(+)

View commit history
git log

Output:
commit a1b2c3d4e5f67890
Author: Your Name <your@email.com>
Date: Mon Jun 15 2026
Initial commit

View commit history (short)
git log --oneline

Output:
a1b2c3d Initial commit

Create a new branch
git branch branch_name

List all branches
git branch

Output:
* master
branch_name

Switch to a branch
git checkout branch_name

Output:
Switched to branch 'branch_name'

Create and switch to a new branch
git checkout -b branch_name

Output:
Switched to a new branch 'branch_name'

Merge a branch into current branch
git merge branch_name

Output:
Merge made by the 'ort' strategy.

Clone a repository
git clone repository_url

Output:
Cloning into 'repository'...
Receiving objects: 100%
Done.

Add remote repository
git remote add origin repository_url

View remote repositories
git remote -v

Output:
origin https://github.com/user/repo.git (fetch)
origin https://github.com/user/repo.git (push)

Push code to GitHub
git push origin main

Output:
Enumerating objects: 5
Writing objects: 100%
To https://github.com/user/repo.git

Pull latest changes
git pull origin main

Output:
Already up to date.

Fetch latest changes
git fetch

Output:
Fetching origin

Remove file from staging area
git reset filename

Output:
Unstaged changes after reset

Delete a branch
git branch -d branch_name

Output:
Deleted branch branch_name (was a1b2c3d).

Practical 2: Create a Fork Repository in GitHub and Apply Branch, Merge, and Rebase Concepts

Commands:

#Clone the forked repository

```
git clone https://github.com/your-username/repository-name.git
```

Output:

Cloning into 'repository-name'...

Receiving objects: 100%

Resolving deltas: 100%

Done.

#Move into repository

```
cd repository-name
```

#Create and switch to a new branch

```
git checkout -b feature-branch
```

Output:

Switched to a new branch 'feature-

branch'

#Create a file and commit changes

```
echo "Feature Added" > feature.txt
```

```
git add .
```

```
git commit -m "Added feature"
```

Output:

[feature-branch a1b2c3d] Added feature

1 file changed, 1 insertion(+)

create mode 100644 feature.txt

#Push branch to GitHub

```
git push origin feature-branch
```

Output:

Enumerating objects: 3

Writing objects: 100%

To <https://github.com/your-username/repository-name.git>

* [new branch] feature-branch -> feature-branch

#Merge branch into main

```
git checkout main
```

```
git merge feature-branch
```

```
git push origin main
```

Output:

Updating e5f6g7h..a1b2c3d

Fast-forward

feature.txt | 1 +

1 file changed, 1 insertion(+)

To <https://github.com/your-username/repository-name.git>

e5f6g7h..a1b2c3d main -> main

#Create another branch for ebase

```
git checkout -b rebase-branch
```

Output:

Switched to a new branch 'rebase-branch'

#Make changes and commit

```
echo "Rebase Example" > rebase.txt
```

```
git add .
```

```
git commit -m "Added rebase example"
```

Output:

[rebase-branch b2c3d4e] Added rebase example

1 file changed, 1 insertion(+)

create mode 100644 rebase.txt

#Rebase branch with main

```
git rebase main
```

Output:

Successfully rebased and

updated

refs/heads/rebase-branch.

#Merge rebased branch into main

```
git checkout main
```

```
git merge rebase-branch
```

Output:

Switched to branch 'main'

Updating a1b2c3d..b2c3d4e

Fast-forward

rebase.txt | 1 +

1 file changed, 1 insertion(+)

#Push final

```
changes git push
```

origin main

Output:

To [https://github.co](https://github.com/your-username/repository-name.git)

[m/your-](https://github.com/your-username/repository-name.git)

[username/repositor](https://github.com/your-username/repository-name.git)

[y-name.git](https://github.com/your-username/repository-name.git)

a1b2c3d..b2c3d4e

#View commit history

```
git log --oneline --graph --all
```

Output:

* b2c3d4e Added rebase example

* a1b2c3d Added feature * e5f6g7h Initial commit

Practical 3: Using Git for Collaboration

Commands:

Topic: Clone Shared Repository

```
git clone https://github.com/username/repository.git
```

Output:

```
Cloning into 'repository'...  
Receiving objects: 100%  
Resolving deltas: 100%  
Done.
```

Topic: Move into Repository

```
cd repository
```

Topic: Create a New Branch

```
git checkout -b feature-branch
```

Output:

```
Switched to a new branch 'feature-branch'
```

Topic: Modify Project Files

(Make changes to project files)

Topic: Check Repository Status

```
git status
```

Output:

```
On branch feature-branch  
Changes not staged for commit:  
  modified: project_file.txt
```

Topic: Add Changes to Staging Area

```
git add .
```

Topic: Commit Changes

```
git commit -m "Added new feature"
```

Output:

```
[feature-branch c3d4e5f] Added new feature
```

1 file changed, 5 insertions(+)

Topic: Push Branch to GitHub

git push origin feature-branch

Output:

Enumerating objects: 4

Writing objects: 100%

To https://github.com/username/repository.git

* [new branch] feature-branch -> feature-branch

Topic: Switch to Main Branch

git checkout main

Output:

Switched to branch 'main'

Topic: Pull Latest Changes

git pull origin main

Output:

From https://github.com/username/repository.git

* branch main -> FETCH_HEAD

Already up to date.

Topic: Merge Feature Branch

git merge feature-branch

Output:

Updating a1b2c3d..c3d4e5f

Fast-forward

project_file.txt | 5 +++++

1 file changed, 5 insertions(+)

Topic: Push Merged Changes

git push origin main

Output:

To https://github.com/username/repository.git

a1b2c3d..c3d4e5f main -> main

Topic: View Commit History

git log --oneline --graph

Output:

* c3d4e5f Added new feature

* a1b2c3d Initial commit

Practical 4: Collaboration and Cloning Using GitHub

Commands:

Topic: Clone Repository from GitHub

```
git clone https://github.com/username/repository.git
```

Output:

Cloning into 'repository'...

Receiving objects: 100%

Resolving deltas: 100%

Done.

Topic: Move into Repository Directory

```
cd repository
```

Topic: Check Repository Status

```
git status
```

Output:

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Topic: Create a New Branch

```
git checkout -b feature-branch
```

Output:

Switched to a new branch 'feature-branch'

Topic: Modify Project Files

(Make changes to project files)

Topic: Add Changes to Staging Area

```
git add .
```

Topic: Commit Changes

```
git commit -m "Added new feature"
```

Output:

[feature-branch d4e5f6g] Added new feature

1 file changed, 8 insertions(+)

Topic: Push Branch to GitHub

```
git push origin feature-branch
```

Output:

Enumerating objects: 5

Writing objects: 100%

To https://github.com/username/repository.git

* [new branch] feature-branch -> feature-branch

Topic: Get Latest Updates from Remote Repository

```
git pull origin main
```

Output:

From https://github.com/username/repository.git

* branch main -> FETCH_HEAD

Already up to date.

Topic: Switch to Main Branch

```
git checkout main
```

Output:

Switched to branch 'main'

Topic: Merge Feature Branch

```
git merge feature-branch
```

Output:

Updating a1b2c3d..d4e5f6g

Fast-forward

project_file.txt | 8 +++++++

1 file changed, 8 insertions(+)

Topic: Push Merged Changes

```
git push origin main
```

Output:

To https://github.com/username/repository.git

a1b2c3d..d4e5f6g main -> main

Topic: View Commit History

```
git log --oneline --graph --all
```

Output:

* d4e5f6g Added new feature

* a1b2c3d Initial commit

Practical 5: Using gitlab web ide

Procedure / Commands:

1. Login to GitLab.
2. Open the desired project/repository.
3. Click Web IDE from the repository page.
4. Create or edit files using the Web IDE.
5. Save the changes.
6. Enter a commit message.
7. Commit changes to a new branch.
8. Create a Merge Request (MR).
9. Review and merge the Merge Request into the main branch.
10. Verify the changes in the repository.

Git Commands (Equivalent):

Topic: Clone Repository

```
git clone https://gitlab.com/username/project.git
```

Output:

```
Cloning into 'project'...  
Receiving objects: 100%  
Resolving deltas: 100%  
Done.
```

Topic: Create a New Branch

```
git checkout -b feature-branch
```

Output:

```
Switched to a new branch 'feature-branch'
```

Topic: Modify Project Files in GitLab Web IDE

(Make changes to project files)

Topic: Add Changes to Staging Area

```
git add .
```

Topic: Commit Changes

```
git commit -m "Updated project files"
```

Output:

```
[feature-branch e5f6g7h] Updated project files  
1 file changed, 6 insertions(+)
```

Topic: Push Changes to GitLab

```
git push origin feature-branch
```

Output:

Enumerating objects: 4

Writing objects: 100%

To <https://gitlab.com/username/project.git>

* [new branch] feature-branch -> feature-branch

Topic: Switch to Main Branch

```
git checkout main
```

Output:

Switched to branch 'main'

Topic: Merge Feature Branch

```
git merge feature-branch
```

Output:

Updating a1b2c3d..e5f6g7h

Fast-forward

project_file.txt | 6 ++++++

1 file changed, 6 insertions(+)

Practical 6: Performing Merge Request

Using GitLab Procedure:

1. Login to GitLab and open the project repository.
2. Create a new branch from the main branch.
3. Make changes to the project files.
4. Commit and push the changes to the new branch.
5. Navigate to the "Merge Requests" section in GitLab.
6. Click "New Merge Request".
7. Select the source branch and target branch.
8. Review the changes and add a title and description.
9. Submit the Merge Request.
10. Review and approve the Merge Request.
11. Click "Merge" to merge the changes into the target branch.

Commands:

Topic: Clone Repository

```
git clone https://gitlab.com/username/project.git
```

Output:

Cloning into 'project'...

Receiving objects: 100%

Resolving deltas: 100%

Done.

Topic: Navigate to Repository

```
cd project
```

Topic: Create a New Branch

```
git checkout -b feature-branch
```

Output:

Switched to a new branch 'feature-branch'

Topic: Modify Project Files

(Make changes and save files)

Output:

(no output)

Topic: Add Files to Staging Area

```
git add .
```

Topic: Commit Changes

```
git commit -m "Added new feature"
```

Output:

```
[feature-branch a1b2c3d] Added new feature  
1 file changed, 8 insertions(+)
```

Topic: Push Branch to GitLab

```
git push origin feature-branch
```

Output:

```
Enumerating objects: 4  
Writing objects: 100%  
To https://gitlab.com/username/project.git  
* [new branch] feature-branch -> feature-branch
```

Topic: Create Merge Request in GitLab

Output:

```
Merge Request created successfully.  
Source Branch: feature-branch  
Target Branch: main
```

Topic: Merge Merge Request through GitLab Interface

Output:

```
Merge request accepted.  
Changes successfully merged into 'main'.
```

Topic: View Commit History

```
git log --oneline --graph --all
```

Output:

```
* a1b2c3d Added new feature  
* e5f6g7h Initial commit
```


Practical 7: Workflow management in Gitlab

Procedure:

1. Login to GitLab and open the project repository.
2. Create a feature branch from the main branch.
3. Make changes to the project files.
4. Commit and push the changes to the feature branch.
5. Create a Merge Request (MR) for code review.
6. Review and approve the Merge Request.
7. Merge the feature branch into the main branch.
8. Trigger and monitor the CI/CD pipeline.
9. Verify successful deployment and integration.

Commands:

Topic: Clone Repository

```
git clone https://gitlab.com/username/project.git
```

Output:

```
Cloning into 'project'...  
Receiving objects: 100%  
Resolving deltas: 100%  
Done.
```

Topic: Navigate to Repository

```
cd project
```

Topic: Create a Feature Branch

```
git checkout -b feature-branch
```

Output:

```
Switched to a new branch 'feature-branch'
```

Topic: Check Repository Status

```
git status
```

Output:

```
On branch feature-branch  
nothing to commit, working tree clean
```

Topic: Add Changes to Staging Area

```
git add .
```

Topic: Commit Changes

```
git commit -m "Implemented new feature"
```

Output:

```
[feature-branch f6g7h8i] Implemented new feature  
1 file changed, 10 insertions(+)
```

Topic: Push Branch to GitLab

```
git push origin feature-branch
```

Output:

```
Enumerating objects: 5  
Writing objects: 100%  
To https://gitlab.com/username/project.git  
* [new branch] feature-branch -> feature-branch
```

Topic: Pull Latest Changes

```
git pull origin main
```

Output:

```
From https://gitlab.com/username/project.git  
* branch      main -> FETCH_HEAD  
Already up to date.
```

Topic: Switch to Main Branch

```
git checkout main
```

Output:

```
Switched to branch 'main'
```

Topic: Merge Feature Branch

```
git merge feature-branch
```

Output:

```
Updating a1b2c3d..f6g7h8i  
Fast-forward  
 project_file.txt | 10 ++++++++  
1 file changed, 10 insertions(+)
```

Topic: Push Merged Code

```
git push origin main
```

Output:

```
To https://gitlab.com/username/project.git  
a1b2c3d..f6g7h8i main -> main
```

Topic: View Commit History

```
git log --oneline --graph --all
```

Output:

- * f6g7h8i Implemented new feature

- * a1b2c3d Initial commit

Practical 8: Demonstrate Continuous Integration and Continuous Deployment (CI/CD) Using Jenkins

Procedure:

1. Install and start Jenkins.
2. Open Jenkins in a web browser.
3. Create a new Freestyle Project or Pipeline.
4. Connect Jenkins with a Git repository.
5. Configure build triggers (e.g., Poll SCM or Webhook).
6. Add build steps.
7. Save and run the job.
8. Verify that Jenkins automatically builds the project after code changes.
9. Configure deployment steps.
10. Monitor build and deployment status through the Jenkins dashboard.

Commands:

Check Java installation

```
java -version
```

Start Jenkins (Linux)

```
sudo systemctl start jenkins
```

Check Jenkins status

```
sudo systemctl status jenkins
```

Clone repository

```
git clone https://github.com/username/repository.git
```

Navigate to project directory

```
cd repository
```

Jenkins Pipeline Example

```
pipeline {  
  agent any  
  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building Application'      }  
    }  
  }  
}
```

```

    }

}

stage('Test'){

    steps {

        echo 'Running Tests'

    }

}

stage('Deploy'){

    steps {

        echo 'Deploying Application'

    }

}

}

}

```

Output:

1. Check Java installation <pre>student@ubuntu:~\$ java -version openjdk version "17.0.10" 2024-01-16 OpenJDK Runtime Environment (build 17.0.10+7-Ubuntu-0ubuntu122.04) OpenJDK 64-Bit Server VM (build 17.0.10+7-Ubuntu-0ubuntu122.04, mixed mode, sharing)</pre>			
2. Start Jenkins (Linux) <pre>student@ubuntu:~\$ sudo systemctl start jenkins (no output)</pre>			
3. Check Jenkins status <pre>student@ubuntu:~\$ sudo systemctl status jenkins ● jenkins.service - Jenkins Continuous Integration Server Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset: enabled) Active: active (running) since Sat 25 May 2024 10:15:42 AM IST; 2s ago Main PID: 12345 (java) Tasks: 62 (limit: 4571) Memory: 512.3M CPU: 8.742s CGroup: /system.slice/jenkins.service └─12345 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war</pre>			
4. Clone repository <pre>student@ubuntu:~\$ git clone https://github.com/username/repository.git Cloning into 'repository'... remote: Enumerating objects: 215, done. remote: Counting objects: 100% (215/215), done. remote: Compressing objects: 100% (120/120), done. remote: Total 215 (delta 53), reused 203 (delta 45), pack-reused 0 Receiving objects: 100% (215/215), 2.34 MiB 1.23 MiB/s, done. Resolving deltas: 100% (53/53), done.</pre>			
5. Navigate to project directory <pre>student@ubuntu:~\$ cd repository student@ubuntu:~/repository\$ ls README.md Jenkinsfile src pom.xml</pre>			
6. Jenkins Pipeline Execution	<table border="0"> <tr> <td> ✓ Start of Pipeline ✓ Build ✓ Test ✓ Deploy ✓ End of Pipeline </td><td> <pre>Started by user admin [Pipeline] Start of Pipeline [Pipeline] stage [Pipeline] { (Build) Building Application [Pipeline] } [Pipeline] stage [Pipeline] { (Test) Running Tests [Pipeline] } [Pipeline] stage [Pipeline] { (Deploy) Deploying Application [Pipeline] } [Pipeline] End of Pipeline Finished: SUCCESS</pre> </td></tr> </table>	✓ Start of Pipeline ✓ Build ✓ Test ✓ Deploy ✓ End of Pipeline	<pre>Started by user admin [Pipeline] Start of Pipeline [Pipeline] stage [Pipeline] { (Build) Building Application [Pipeline] } [Pipeline] stage [Pipeline] { (Test) Running Tests [Pipeline] } [Pipeline] stage [Pipeline] { (Deploy) Deploying Application [Pipeline] } [Pipeline] End of Pipeline Finished: SUCCESS</pre>
✓ Start of Pipeline ✓ Build ✓ Test ✓ Deploy ✓ End of Pipeline	<pre>Started by user admin [Pipeline] Start of Pipeline [Pipeline] stage [Pipeline] { (Build) Building Application [Pipeline] } [Pipeline] stage [Pipeline] { (Test) Running Tests [Pipeline] } [Pipeline] stage [Pipeline] { (Deploy) Deploying Application [Pipeline] } [Pipeline] End of Pipeline Finished: SUCCESS</pre>		

Practical 9: Explore Docker Commands for Container Management

Commands:

Check Docker version

```
docker --version
```

Display Docker system information

```
docker info
```

Download an image from Docker Hub

```
docker pull ubuntu
```

List downloaded images

```
docker images
```

Create and run a container

```
docker run -it ubuntu
```

Run container in detached mode

```
docker run -d nginx
```

List running containers

```
docker ps
```

List all containers

```
docker ps -a
```

Stop a container

```
docker stop <container_id>
```

Start a container

```
docker start <container_id>
```

Restart a container

```
docker restart <container_id>
```

Access a running container

```
docker exec -it <container_id> bash
```

View container logs

```
docker logs <container_id>
```

Remove a container

```
docker rm <container_id>
```

Remove an image

```
docker rmi <image_id>
```

List Docker networks

```
docker network ls
```

List Docker volumes

```
docker volume ls
```

View Docker processes

```
docker top <container_id>
```

View resource usage

```
docker stats
```

Output:

```
1. Check Docker version
user@ubuntu:~$ docker --version
Docker version 27.0.3, build abc1234

2. Display Docker system information
user@ubuntu:~$ docker info
Containers: 5
  Running: 2
  Paused: 0
  Stopped: 3
Images: 8
Server Version: 27.0.3
Storage Driver: overlay2

3. Download an image from Docker Hub
user@ubuntu:~$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Status: Downloaded newer image for ubuntu:latest

4. List downloaded images
user@ubuntu:~$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
ubuntu       latest   a04dc4851cbc   2 weeks ago    78MB
nginx        latest   605c77e624dd   3 weeks ago    192MB

5. Create and run a container
user@ubuntu:~$ docker run -it ubuntu
root@7f8a9b3c2d1e:/# exit
exit

6. Run container in detached mode
user@ubuntu:~$ docker run -d nginx
8d3f5e7c9b12f6a4d8c7e5f9a1b2c3d4e5f67890

7. List running containers
user@ubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
8d3f5e7c9b12   nginx     "/docker-entrypoint.."   12 seconds ago Up 10 seconds  80/tcp       nifty_ramanujan

8. List all containers
user@ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
8d3f5e7c9b12   nginx     "/docker-entrypoint.."   25 seconds ago Up 10 seconds  80/tcp       nifty_ramanujan
7f8a9b3c2d1e   ubuntu    "bash"                   1 minute ago   Exited (0) 30 seconds ago   zen_cray

9. Stop a container
user@ubuntu:~$ docker stop 8d3f5e7c9b12
8d3f5e7c9b12

10. Start a container
user@ubuntu:~$ docker start 8d3f5e7c9b12
8d3f5e7c9b12

11. Restart a container
user@ubuntu:~$ docker restart 8d3f5e7c9b12
8d3f5e7c9b12

12. Access a running container
user@ubuntu:~$ docker exec -it 8d3f5e7c9b12 bash
root@8d3f5e7c9b12:/# exit
exit

13. View container logs
user@ubuntu:~$ docker logs 8d3f5e7c9b12
2026/06/15 10:20:01 [notice] 1#1: using the "epoll" event method
2026/06/15 10:20:01 [notice] 1#1: nginx/1.25.1
2026/06/15 10:20:01 [notice] 1#1: start worker processes
2026/06/15 10:20:01 [notice] 1#1: start worker process 31

14. Remove a container
user@ubuntu:~$ docker rm 7f8a9b3c2d1e
7f8a9b3c2d1e

15. Remove an image
user@ubuntu:~$ docker rmi a04dc4851cbc
Untagged: ubuntu:latest
Deleted: sha256:a04dc4851cbc...

16. List Docker networks
user@ubuntu:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge   bridge    local
b2c3d4e5f6g7   host     host      local
c3d4e5f6g7h8   none     none      local

17. List Docker volumes --
user@ubuntu:~$ docker volume ls
DRIVER    VOLUME NAME
local     myvolume
local     nginx_data

18. View Docker processes --
user@ubuntu:~$ docker top 8d3f5e7c9b12
UID        PID        PPID       C    STIME     TTY      TIME          CMD
root       1234       1          0    10:20    ?        00:00:00    nginx: master process nginx -g daemon off;
root       1235       1          0    10:20    ?        00:00:00    nginx: worker process

19. View resource usage
user@ubuntu:~$ docker stats --no-stream
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
8d3f5e7c9b12   nifty_ramanujan 0.12%     18.23MiB / 2GiB     0.89%     1.21kB / 3.47kB    0B / 0B     6
```


Practical 10: Develop a Simple Containerized Application Using Docker

Requirements:

- Docker Installed
- Python Installed

Application Code (app.py):

```
from flask import Flask

app = Flask(__name__)

@app.route('/')

def home():

    return "Hello from Docker Container!"

if __name__ == "__main__":

    app.run(host='0.0.0.0', port=5000)
```

Requirements File (requirements.txt):

```
flask
```

Dockerfile:

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY . /app
```

```
RUN pip install -r requirements.txt
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Commands:

Build Docker image

```
docker build -t flask-app .
```

List images

```
docker images
```

Run container

```
docker run -d -p 5000:5000 flask-app
```

Check running containers

`docker ps`

View logs

`docker logs <container_id>`

Stop container

`docker stop <container_id>`

Remove container

`docker rm <container_id>`

Output:

```
1. Environment Check
$ docker --version
Docker version 27.0.3, build a5d2c30
$ python --version
Python 3.9.18

2. Application Files Created
$ ls
app.py  requirements.txt  Dockerfile

3. Build Docker image
$ docker build -t flask-app .
Sending build context to Docker daemon 5.12kB
Step 1/5 : FROM python:3.9
--> a7b0c4f61e8b
Step 2/5 : WORKDIR /app
--> Running in 12ab34cd56ef
--> 7d2f3e4b5a6c
Step 3/5 : COPY . /app
--> 3c4d5e6f7a8b
Step 4/5 : RUN pip install -r requirements.txt
--> Running in 9f8e7d6c5b4a
Collecting flask
Successfully installed flask-3.0.3
Step 5/5 : CMD ["python", "app.py"]
--> Running in 0a1b2c3d4e5f
--> f3d2c1b4a5e6
Successfully built f3d2c1b4a5e6
Successfully tagged flask-app:latest

4. List Docker images
$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
flask-app    latest    f3d2c1b4a5e6   2 minutes ago  1.02GB
python       3.9       a7b0c4f61e8b   3 weeks ago    1.01GB

5. Run container
$ docker run -d -p 5000:5000 flask-app
8d3f5e7c9b12f6a4d8c7e5f9a1b2c3d4e5f67890

6. Check running containers
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
8d3f5e7c9b12   flask-app "python app.py"         10 seconds ago Up 9 seconds   0.0.0.0:5000->5000/tcp

7. Access application in browser
URL: http://localhost:5000
Output:
Hello from Docker Container!

8. View logs
$ docker logs 8d3f5e7c9b12
* Serving Flask app 'app'
* Debug mode: off
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit

9. Stop container
$ docker stop 8d3f5e7c9b12
8d3f5e7c9b12

10. Remove container
$ docker rm 8d3f5e7c9b12
8d3f5e7c9b12
```

Practical 11: Ad-hoc ansible commands Prerequisites:

- **Ansible Installed**
- **Inventory file configured with target hosts**

Commands:

Check connectivity with all hosts

```
ansible all -m ping
```

Display hostname of remote hosts

```
ansible all -a "hostname"
```

Check disk usage

```
ansible all -a "df -h"
```

Check memory usage

```
ansible all -a "free -m"
```

Create a directory

```
ansible all -m file -a "path=/tmp/demo state=directory"
```

Create a file

```
ansible all -m file -a "path=/tmp/demo/test.txt state=touch"
```

Copy a file to remote hosts

```
ansible all -m copy -a "src=test.txt dest=/tmp/test.txt"
```

Install a package

```
ansible all -m package -a "name=httpd state=present"
```

Start a service

```
ansible all -m service -a "name=httpd state=started"
```

Stop a service

```
ansible all -m service -a "name=httpd state=stopped"
```

Restart a service

```
ansible all -m service -a "name=httpd state=restarted"
```

Gather system information

ansible all -m setup

Check uptime

ansible all -a "uptime"

Output:

```
1. Check connectivity with all hosts
controlnode:~$ ansible all -m ping
webserver1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
webserver2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}

-----

2. Display hostname of remote hosts
controlnode:~$ ansible all -a "hostname"
webserver1 | CHANGED | rc=0 >>
webserver1

webserver2 | CHANGED | rc=0 >>
webserver2

-----

3. Check disk usage
controlnode:~$ ansible all -a "df -h"
webserver1 | CHANGED | rc=0 >>
webserver2 | CHANGED | rc=0 >>
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   12G   36G   25% /
webserver2 | CHANGED | rc=0 >>
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   10G   38G   22% /

-----

4. Check memory usage
controlnode:~$ ansible all -a "free -m"
webserver1 | CHANGED | rc=0 >>
webserver2 | CHANGED | rc=0 >>
Mem:            total    used    free   shared buff/cache   available
Mem:            4096    1450    2646      74      350     2320
webserver2 | CHANGED | rc=0 >>
Mem:            4096    1380    2716      72      320     2390

-----

5. Create a directory
controlnode:~$ ansible all -m file -a "path=/tmp/demo state=directory"
webserver1 | SUCCESS => {"changed": true, "path": "/tmp/demo", "state": "directory"}
webserver2 | SUCCESS => {"changed": true, "path": "/tmp/demo", "state": "directory"}

-----

6. Create a file
controlnode:~$ ansible all -m file -a "path=/tmp/demo/test.txt state=touch"
webserver1 | SUCCESS => {"changed": true, "dest": "/tmp/demo/test.txt"}
webserver2 | SUCCESS => {"changed": true, "dest": "/tmp/demo/test.txt"}

-----

7. Copy a file to remote hosts
controlnode:~$ ansible all -m copy -a "src=test.txt dest=/tmp/test.txt"
webserver1 | SUCCESS => {"changed": true, "dest": "/tmp/test.txt"}
webserver2 | SUCCESS => {"changed": true, "dest": "/tmp/test.txt"}

-----

8. Install a package
controlnode:~$ ansible all -m package -a "name=httpd state=present"
webserver1 | SUCCESS => {"changed": true, "msg": "Package httpd-2.4.58 installed"}
webserver2 | SUCCESS => {"changed": true, "msg": "Package httpd-2.4.58 installed"}

-----

9. Start a service
controlnode:~$ ansible all -m service -a "name=httpd state=started"
webserver1 | SUCCESS => {"changed": true, "state": "started"}
webserver2 | SUCCESS => {"changed": true, "state": "started"}

-----

10. Stop a service
controlnode:~$ ansible all -m service -a "name=httpd state=stopped"
webserver1 | SUCCESS => {"changed": true, "state": "stopped"}
webserver2 | SUCCESS => {"changed": true, "state": "stopped"}

-----

11. Restart a service
controlnode:~$ ansible all -m service -a "name=httpd state=restarted"
webserver1 | SUCCESS => {"changed": true, "state": "restarted"}
webserver2 | SUCCESS => {"changed": true, "state": "restarted"}

-----

12. Gather system information
controlnode:~$ ansible all -m setup
webserver1 | SUCCESS => {
  "ansible_hostname": "webserver1",
  "ansible_distribution": "Ubuntu",
  "ansible_distribution_version": "22.04",
  "ansible_kernel": "5.15.0-94-generic",
  "ansible_architecture": "x86_64"
} ... (output truncated)
webserver2 | SUCCESS => {
  "ansible_hostname": "webserver2",
  "ansible_distribution": "Ubuntu",
  "ansible_distribution_version": "22.04",
  "ansible_kernel": "5.15.0-94-generic",
  "ansible_architecture": "x86_64"
} ... (output truncated)

-----

13. Check uptime
controlnode:~$ ansible all -a "uptime"
webserver1 | CHANGED | rc=0 >>
14:25:31 up 5 days, 8:42, 2 users, load average: 0.15, 0.10, 0.08
webserver2 | CHANGED | rc=0 >>
14:25:31 up 5 days, 8:40, 1 user, load average: 0.12, 0.09, 0.07
```

Practical 12: Using ansible playbooks

Inventory File (hosts):

[servers]

server1 ansible_host=192.168.1.10

server2 ansible_host=192.168.1.11

Playbook (playbook.yml):

- name: Install and Start Apache Web Server

hosts: servers

become: yes

tasks:

- name: Install Apache

package:

name: httpd

state: present

- name: Start Apache Service

service:

name: httpd

state: started

enabled: yes

Commands:

Check syntax of playbook

ansible-playbook playbook.yml --syntax-check

Execute playbook

ansible-playbook -i hosts playbook.yml

Execute playbook with privilege escalation

ansible-playbook -i hosts playbook.yml --become

Dry run (check mode)

ansible-playbook -i hosts playbook.yml --check

List hosts targeted by playbook

ansible-playbook -i hosts playbook.yml --list-hosts

View detailed output

ansible-playbook -i hosts playbook.yml -v

Output:

1. Inventory File (hosts)

```
[servers]
server1 ansible_host=192.168.1.10
server2 ansible_host=192.168.1.11
```

2. Playbook (playbook.yml)

```
- name: Install and Start Apache Web Server
  hosts: servers
  become: yes
  tasks:
    - name: Install Apache package
      package:
        name: httpd
        state: present
    - name: Start Apache Service
      service:
        name: httpd
        state: started
        enabled: yes
```

3. Check syntax of playbook

```
student@ansible:~$ ansible-playbook playbook.yml --syntax-check
playbook: playbook.yml
```

4. Execute playbook

```
student@ansible:~$ ansible-playbook -i hosts playbook.yml

PLAY [Install and Start Apache Web Server] *****
TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
TASK [Install Apache package] *****
changed: [server1]
changed: [server2]
TASK [Start Apache Service] *****
changed: [server1]
changed: [server2]
PLAY RECAP *****
server1      : ok=3  changed=2  unreachable=0  failed=0
server2      : ok=3  changed=2  unreachable=0  failed=0
```

5. Execute playbook with privilege escalation

```
student@ansible:~$ ansible-playbook -i hosts playbook.yml --become

PLAY [Install and Start Apache Web Server] *****
TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
TASK [Install Apache package] *****
ok: [server1]
ok: [server2]
TASK [Start Apache Service] *****
ok: [server1]
ok: [server2]
PLAY RECAP *****
server1      : ok=3  changed=0  unreachable=0  failed=0
server2      : ok=3  changed=0  unreachable=0  failed=0
```

6. Dry run (check mode)

```
student@ansible:~$ ansible-playbook -i hosts playbook.yml --check

PLAY [Install and Start Apache Web Server] *****
TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
TASK [Install Apache package] *****
changed: [server1]
changed: [server2]
TASK [Start Apache Service] *****
changed: [server1]
changed: [server2]
PLAY RECAP *****
server1      : ok=3  changed=2  unreachable=0  failed=0
server2      : ok=3  changed=2  unreachable=0  failed=0
```

7. List hosts targeted by playbook

```
student@ansible:~$ ansible-playbook -i hosts playbook.yml --list-hosts

playbook: playbook.yml

play #1 (servers): Install and Start Apache Web Server
hosts (2):
    server1
    server2
```

8. View detailed output

```
student@ansible:~$ ansible-playbook -i hosts playbook.yml -v

Using /etc/ansible/ansible.cfg as config file
TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
TASK [Install Apache package] *****
ok: [server1]
ok: [server2]
TASK [Start Apache Service] *****
ok: [server1]
ok: [server2]
PLAY RECAP *****
server1      : ok=3  changed=0  unreachable=0  failed=0
server2      : ok=3  changed=0  unreachable=0  failed=0
```

Summary of Commands	
Sr.No.	Command
1	ansible-playbook playbook.yml --syntax-check
2	ansible-playbook -i hosts playbook.yml
3	ansible-playbook -i hosts playbook.yml --become
4	ansible-playbook -i hosts playbook.yml --check
5	ansible-playbook -i hosts playbook.yml --list-hosts
6	ansible-playbook -i hosts playbook.yml -v

